Sprint Review — Team L Sprint 1 Review

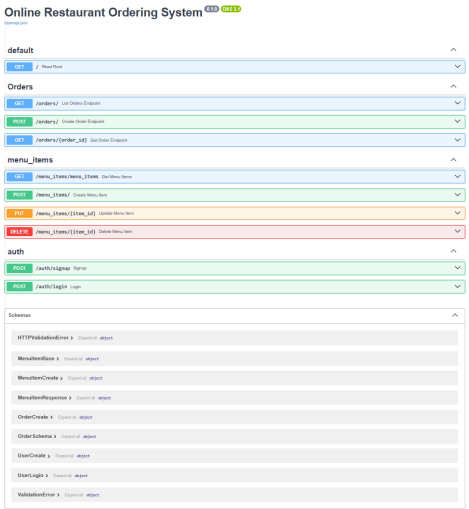
Sprint goals

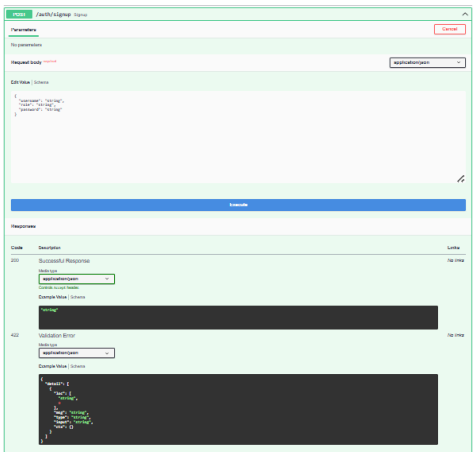
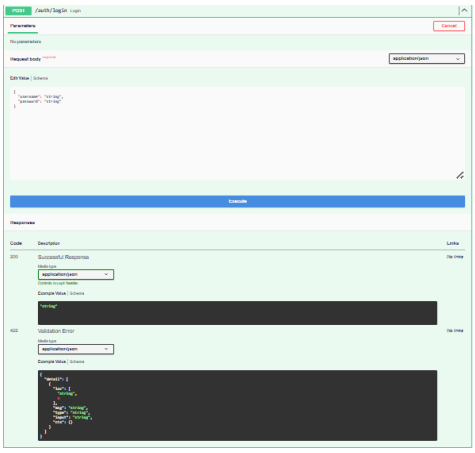
My goal for Sprint 1 was to independently build and verify the backend CRUD functionality and establish the authentication groundwork for the Final Project. I focused on creating stable endpoints for menu item management, user account creation, and database connectivity. By the end of the sprint, I successfully validated all CRUD operations in Swagger UI and implemented a working JWT-based authentication system.

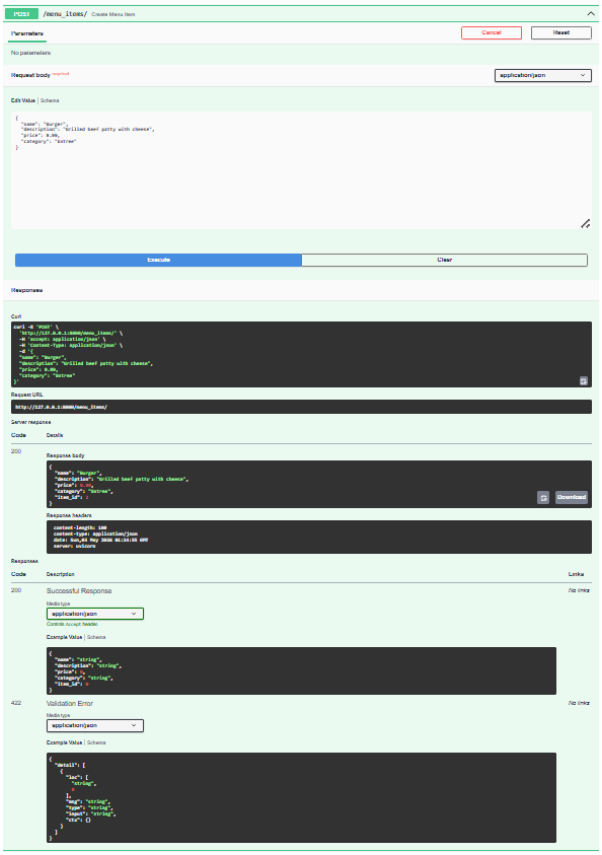
<u>Backend CRUD Setup and Authentication Groundwork</u>

- Key result 1 - I successfully implemented and tested all CRUD endpoints for menu items in FastAPI, verifying each operation (POST, GET, PUT, DELETE) through Swagger UI with 200 OK and 204 No Content responses.
- Key result 2 - I configured the database connection and finalized the schema alignment between SQLAlchemy models and Pydantic schemas, ensuring consistent data storage and retrieval for menu items and users.
- Key result 3 - I developed and validated the authentication system using JWT tokens, completing both the Signup and Login endpoints and confirming secure token generation and user verification.

Demos

Demo	Feedback
Work item you are demoing	Questions and comments from the team
CRUD operations in Swagger UI  	All endpoints executed successfully; no errors

Login/Signup demo  	Token generated correctly; ready for security integration in Sprint 2

Database Integration Demo 	Data stored and retrieved successfully; schema aligned with Pydantic models

PUT /menu_items/{item_id} Update Menu Item

Parameters

Cancel

Reset

Name	Description
item_id * required integer (path)	1

Request body required

application/json

Edit Value | Schema

```
{
  "item_id": 1,
  "name": "Burger",
  "description": "Grilled beef patty with cheese",
  "price": 8.99,
  "category": "Entree"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/menu_items/1' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
  "item_id": 1,
  "name": "Burger",
  "description": "Grilled beef patty with cheese",
  "price": 8.99,
  "category": "Entree"
}'
```

Request URL

http://127.0.0.1:8000/menu_items/1

Server response

Code	Details

200	Response body
	{
	"name": "Burger",
	"description": "Grilled beef patty with cheese",
	"price": 8.99,
	"category": "Entree",
	"item_id": 1
	}
	Download
	Response headers
	content-length: 189
	content-type: application/json
	date: Sun,04 May 2026 01:17:08 GMT
	server: uvicorn

Responses

Code	Description	Links

200	Successful Response
	Media type
	application/json
	Controls Accept header.
	Example Value | Schema
	{
	"name": "string",
	"description": "string",
	"price": 0,
	"category": "string",
	"item_id": 0
	}
		No links

422	Validation Error
	Media type
	application/json
	Example Value | Schema
	{
	"detail": [
	{
	"loc": [
	"string",
	0
	],
	"msg": "string",
	"type": "string",
	"input": "string",
	"ctx": {}
	}
	]
	}
		No links

menu_items

GET /menu_items/menu_items Get Menu Items

Parameters

Cancel

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/menu_items/menu_items' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/menu_items/menu_items

Server response

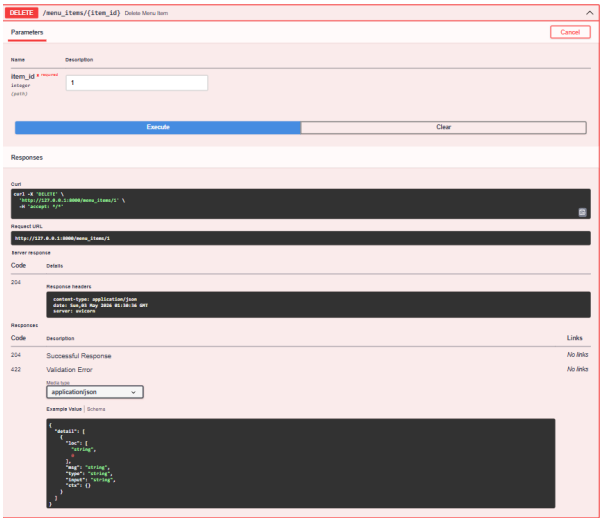
Code	Details

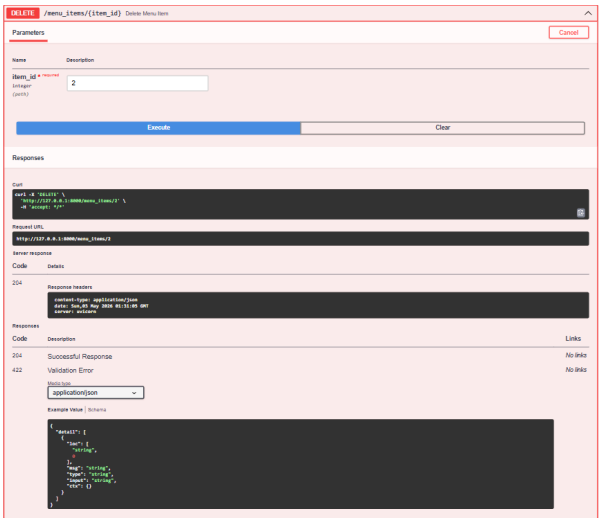
200	Response body
	[
	{
	"name": "Burger",
	"description": "Grilled beef patty with cheese",
	"price": 8.99,
	"category": "Entree",
	"item_id": 1
	},
	{
	"name": "Burger",
	"description": "Grilled beef patty with cheese",
	"price": 8.99,
	"category": "Entree",
	"item_id": 2
	}
	]
	Download
	Response headers
	content-length: 321
	content-type: application/json
	date: Sun,03 May 2026 01:29:29 GMT
	server: uvicorn

Responses

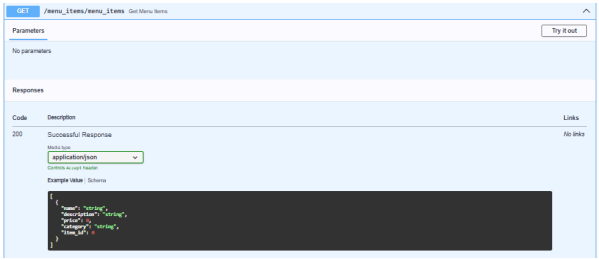
Code	Description	Links

200	Successful Response
	Media type
	application/json
	Controls Accept header.
	Example Value | Schema
	[
	{
	"name": "string",
	"description": "string",
	"price": 0,
	"category": "string",
	"item_id": 0
	}
	]
		No links





API Documentation Demo



Routes organized and tagged; documentation accessible and readable for testing.

POST /menu_items/ Create Menu Item

Parameters

Try it out

No parameters

Request body *required* application/json

Example Value | Schema

```
{
  "name": "string",
  "description": "string",
  "price": 0,
  "category": "string"
}
```

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema	No links
422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "name": "string",
  "description": "string",
  "price": 0,
  "category": "string",
  "item_id": 0
}
```

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

PUT /menu_items/{item_id} Update Menu Item

Parameters

Try it out

Name	Description
item_id * *required* integer (path)	item_id

Request body *required* application/json

Example Value | Schema

```
{
  "name": "string",
  "description": "string",
  "price": 0,
  "category": "string"
}
```

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema	No links
422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "name": "string",
  "description": "string",
  "price": 0,
  "category": "string",
  "item_id": 0
}
```

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

DELETE /menu_items/{item_id} Delete Menu Item

Parameters

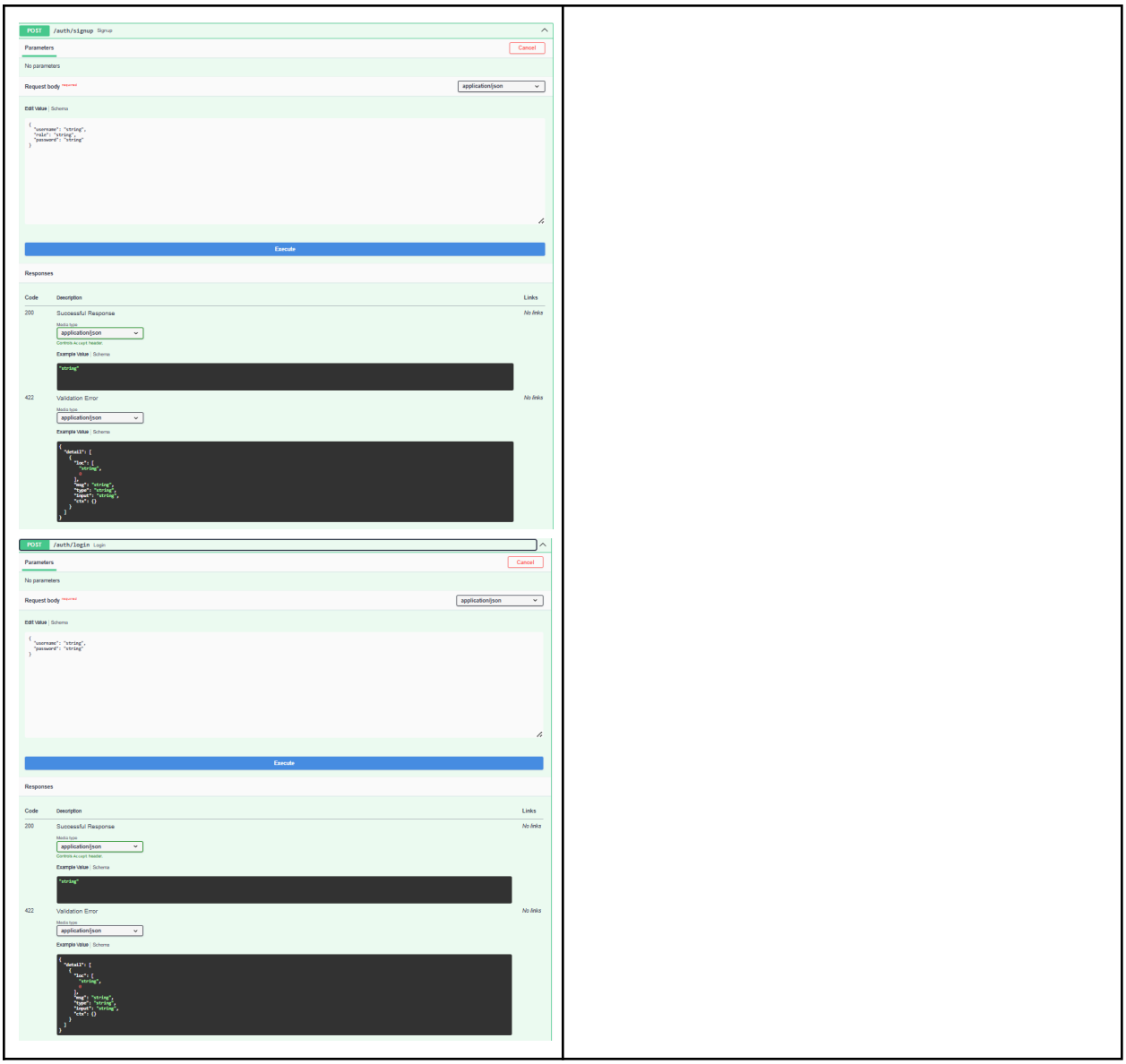
Try it out

Name	Description
item_id * *required* integer (path)	item_id

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```



Complete tasks

During Sprint 1, I completed all backend CRUD endpoints for menu items and users. Each endpoint was tested and validated through Swagger UI with successful responses. I finalized the database schema and implemented the authentication system using JWT tokens. All work items met acceptance criteria and were marked “Done.” This sprint established full backend functionality and data persistence, providing a solid foundation for Sprint 2.

Upcoming

In Sprint 2, I will focus on enhancing security and connecting the backend to the frontend interface. Planned tasks include implementing role-based access control, integrating the login and signup flow with the frontend, adding pagination and filtering to GET /menu_items, and developing automated Pytest cases for endpoint validation. I will also begin preparing deployment documentation and presentation materials for the final project submission.